Tu primer PR en GitHub 🔀

En esta lección aprenderás cómo **subir tu código a `GitHub` por primera vez**, paso a paso. Este es un proceso esencial que repetirás muchas veces en tu carrera como desarrollador, ya que te permitirá **guardar, compartir y colaborar** en tus proyectos desde la nube.

Este es el flujo que usaremos también para subir nuestros ejercicios en `Lyfter`. Es súper importante que estos comandos queden súper claros ya que los ejecutaremos muchas veces durante nuestro paso en `DUAD` y el resto de nuestras carreras.

Guía paso a paso 🧰

1. Crear un repositorio en GitHub 🧱

Aquí crearás la versión remota de tu proyecto.

2. Entra a https://github.com y accede con tu cuenta.
3. Haz clic en el ícono ➕ (*esquina superior derecha*) → "*New repository*".
4. Ponle un nombre, por ejemplo: `Ejercicios_Lyfter`.
5. Selecciona la opción "*Public*".
6. Habilita la opción "*Add README*"
7. Clic en "*Create Repository*".

---

2. Clonar el repositorio localmente

`Clonar` un repositorio crea una copia exacta en tu computadora del repositorio en `GitHub`.

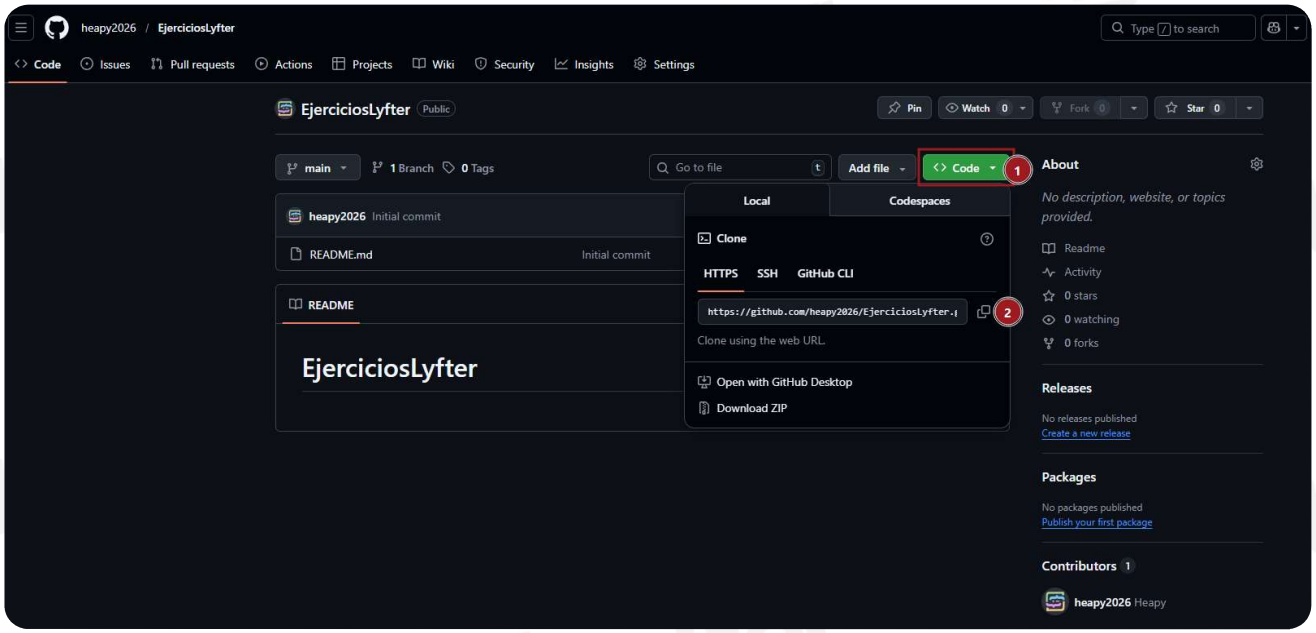
¿Dónde encuentro el enlace del repositorio?

Una vez creado el repositorio en `GitHub`, verás una página con diferentes opciones para clonarlo. Copia la URL que se muestra, por ejemplo:

```
1  https://github.com/tu_usuario/Ejercicios_Lyfter.git
```

Copiar

Imagen referencia para obtener el enlace del repositorio



¿Dónde se va a `clonar` el repositorio en tu computadora?

Antes de ejecutar el comando `git clone`, es importante que estés ubicado en la carpeta donde quieres que se cree el proyecto en tu computadora. Puedes navegar a esa carpeta con el comando `cd`.

Por ejemplo utilizando la terminal, si quieres clonar en tu escritorio:

```
1  cd Desktop
```
Copiar

Puedes usar cualquier carpeta que tengas en tu sistema, lo importante es que **te ubiques allí antes de** `clonar`

Comando `clone` 🤼

Este comando le indica a `Git` que debe descargar una copia exacta del repositorio remoto en tu computadora. Se crea una nueva carpeta con el mismo nombre del repositorio, y dentro de ella estarán todos los archivos y configuraciones.

```
1  git clone https://github.com/tu_usuario/Ejercicios_Lyfter.git
```
Copiar

---

3. Verificar que se clonó correctamente ✅

Este comando te permite **entrar a la carpeta del repositorio que acabas de** `clonar`. Es como abrir una carpeta desde la terminal para poder trabajar dentro de ella.

```
1  cd Ejercicios_Lyfter
```
Copiar

Revisa el estado actual:

Este comando muestra el estado de tu repositorio local, de esta manera podemos saber que el repositorio está listo para funcionar:

```
1  git status
```
Copiar

Salida esperada:

```
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

Copiar

---

4. Crear un nuevo branch 🌿

Antes de agregar nuestros archivos, una excelente práctica es crear un `branch` nuevo, en lugar de trabajar directamente sobre la rama principal (. Esto nos permite realizar nuestros ejercicios en un espacio aislado y es un requisito para poder crear un `Pull request` más adelante.

Entonces, vamos a abrir nuestro editor de texto, como `Visual Studio Code` en la carpeta del repositorio que acabamos de clonar.

Y ahora, para crear este nuevo `branch` y movernos a este automáticamente ejecutamos el siguiente comando:

```
git checkout -b new-branch-name
```

Copiar

Por ejemplo, si queremos agregar los ejercicios de estructuras de control en `Python`, podemos crear el `branch` de la siguiente manera:

```
git checkout -b control-structures-exercises
```

Copiar

💡 A lo largo de la lección seguiremos utilizando este `branch` como ejemplo, si quieres aplicar los comandos, solo debes sustituir por el `branch` que hayas creado y estés trabajando.

5. Agregar archivos al proyecto 🗂️

En `Git`, todo lo que se encuentre **dentro de la carpeta del repositorio** forma parte del proyecto. Esto incluye carpetas, archivos de texto, scripts, imágenes, etc. `Git` se encarga de rastrear los cambios que ocurran dentro de esta carpeta.

Por esta razón, dentro de nuestro repositorio de ejercicios seguimos la siguiente estructura de carpetas:

Copiar

```
Ejercicios_Lyfter/
├── IT_Essentials/
│       ├── Algorithms/
│       │       ├── Algorithm1.txt
│       │       └── Algorithm2.txt
│       │
│       ├── Algorithms_Extra/
│       │       ├── Algorithm1.txt
│       │       └── Algorithm2.txt
│       │
│       ├── Terminals/
│       │       ├── Commands.txt
│       │       └── Commands_Executed.pdf
│       │
│       └── Terminals_Extra/
│               ├── Commands.txt
│               └── Commands_Executed.pdf
│
├── Programming_Logic/
│       ├── Pseudocode/
│       │       ├── Exercise1.txt
│       │       ├── Exercise2.txt
│       │       ├── ...
│       │       └── Exercise10.txt
│       │
│       ├── Pseudocode_Extra/
│       │       ├── Exercise1.txt
│       │       ├── Exercise2.txt
│       │       ├── ...
│       │       └── Exercise10.txt
```

```
31  │       │
32  │       ├── Flowcharts/
33  │       │       ├── Flowchart1.png
34  │       │       ├── Flowchart2.png
35  │       │       ├── ...
36  │       │       └── Flowchart10.png
37  │       │
38  │       └── Flowcharts_Extra/
39  │               ├── Flowchart1.png
40  │               ├── Flowchart2.png
41  │               ├── ...
42  │               └── Flowchart10.png
43  │
44  ├── Python_Basics/
45  │       ├── Intro_to_Python/
46  │       │       ├── exercise1.py
47  │       │       ├── exercise2.py
48  │       │       └── exercise3.py
49  │       │
50  │       ├── Intro_to_Python_Extra/
51  │       │       ├── exercise1.py
52  │       │       ├── exercise2.py
53  │       │       └── exercise3.py
54  │       │
55  │       ├── Syntax/
56  │       │       ├── exercise1.py
57  │       │       ├── exercise2.py
58  │       │       ├── exercise3.py
59  │       │       ├── exercise4.py
60  │       │       └── exercise5.py
```

```
61  │       │
62  │       ├── Syntax_Extra/
63  │       │       ├── exercise1.py
64  │       │       ├── exercise2.py
65  │       │       ├── exercise3.py
66  │       │       ├── exercise4.py
67  │       │       └── exercise5.py
68  │       │
69  │       └── *Incluye el resto de ejercicios...*
70  │
71  └── *Incluye el resto de módulos...*
```

💡 Es muy importante que mantengamos nuestro proyecto bien organizado para que sea fácil de mantener, colaborar y entender más adelante. Como referencia, puedes utilizar el siguiente repositorio: 🔗 Lyfter_Exercises_Structure

---

6. Revisar cambios 🔍

Comando `git status`

Este comando te permite ver el estado actual de tu repositorio. Te muestra qué archivos:

No están siendo seguidos aún por `Git` ("*untracked*").

Han sido modificados pero no agregados ("*modified*").

Están listos para ser confirmados en un `commit` ("*staged*").

Es una herramienta muy útil para revisar tus cambios antes de continuar.

```
git status
```

Copiar

Posibles salidas:

Untracked files: archivos nuevos que `Git` no está siguiendo todavía.

Changes to be committed: archivos listos para ser guardados en un `commit`.

Ejemplo:

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Syntax/
```

Copiar

---

7. Agregar cambios ➕

Comando `git add <archivo>`

Si queremos agregar un archivo a la vez a nuestro próximo `commit`, podemos hacerlo especificando el nombre exacto de nuestro archivo.

```
git add your_file.py
```

Copiar

Comando `git add .`

Por otro lado, el comando con el punto `.` le indica a `Git` que todos los archivos modificados o nuevos deben pasar al área de preparación, también llamada `staging area`. Esto significa que están listos para incluirse en el próximo `commit`.

```
git add .
```

Copiar

Este comando agrega todos los archivos y carpetas modificados o nuevos al área de preparación (`staging`).

Verifica el estado nuevamente:

```
git status
```

Copiar

Salida esperada:

```
Changes to be committed:
        new file:   Syntax/conditionals1.py
```

Copiar

---

8. Hacer el primer commit 📝

Comando `git commit`

Este comando guarda de manera permanente los cambios que has preparado con `git add` . El texto que va entre comillas es un **mensaje descriptivo** que te ayudará a recordar qué cambios contiene ese `commit` .

```
git commit -m "Including Python Syntax exercises"
```

Copiar

Salida esperada:

```
[Ejercicios_Lyfter f34b163] Including Python Syntax exercises
 1 file changed, 1 insertion(+)
 create mode 100644 conditionals1.py
```

Copiar

Buenas prácticas:

Usa mensajes claros y concisos.

Evita mensajes genéricos como "*cambios*".

---

9. Ver el historial 📜

Comando `git log`

Este comando te muestra el historial de cambios del repositorio. Cada vez que haces un `commit`, `Git` guarda un registro con:

Un identificador único (*hash*).

El autor del cambio.

La fecha.

El mensaje del `commit`.

Este historial es útil para revisar cuándo y por qué se hizo cada cambio.

```
git log
```

Copiar

Salida esperada:

```
commit f3e12c9d4a1b2f...
Author: Tu Nombre <tu@email.com>
Date:   Wed May 15 10:00:00 2025 -0600

    Including Python Syntax exercises
```

Copiar

Puedes salir del log presionando `q`.

10. Subir los cambios a GitHub ⬆️

Comando `git push`

Este comando se utiliza para **enviar los cambios confirmados desde tu repositorio local hacia** `GitHub` por primera vez. Veamos qué significa cada parte:

`git push`: envía los cambios al repositorio remoto.

`--set-upstream`: le indica a `Git` que la rama local (*ej.* `control-structures-exercises`) debe estar vinculada a la rama remota del mismo nombre. Esto es útil para que en el futuro podamos simplemente escribir `git push` sin argumentos.

`origin`: es el nombre por defecto del repositorio remoto.

`control-structures-exercises`: es la `rama` en la que estamos trabajando nuestro ejercicio `conditionals1.py` de la carpeta `Syntax`.

```
git push --set-upstream origin control-structures-exercises
```

Copiar

Salida esperada:

```
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Writing objects: 100% (6/6), done.
To <https://github.com/tu_usuario/Ejercicios_Lyfter.git>
 * [new branch]      control-structures-exercises -> control-structures-exercises
```

Copiar

11. Crear el Pull Request 🤝

¿Por qué usamos `Pull requests`?

Un `Pull request`, como su nombre lo dice es una "*petición de unión*". Es decir, en lugar de subir nuestros cambios directamente a la rama principal, usamos esta herramienta para hacer `merge` . De esta manera, otros miembros de nuestro equipo pueden revisar nuestro código, se aseguren que no haya errores y se mantenga la calidad del proyecto. ¡Esta revisión la van a hacer tus revisores!

Una vez que subimos la rama con cambios, seguimos estos pasos:

Ve a la pestaña `Pull requests` en tu repositorio.

Debes seleccionar la rama de destino y de origen:

base: Es la rama destino (*normalmente* `main`), donde queremos que viva nuestro código final.

compare: Es la rama donde subimos nuestros cambios.

Hacemos clic en el botón verde "*Create pull request*"

Agregamos título y descripción:

Título: Debe ser breve y claro (ej. *"Flow control structures exercises"*).

Descripción: Acá detallamos lo que hicimos. (ej.*"Naming, syntax and flow control structures."*).

Crear el PR: Nuevamente hacemos clic en el botón verde "*Create pull request*".

Imagen referencia para crear el PR

Imagen referencia una vez hayamos creado el PR

Una vez finalizado el proceso debes ver esta página y compartir el URL de la misma en tu entregable.

12. Feedback, Merge y Nuevo Entregable 🔄

Recibir Feedback

Una vez que hemos creado nuestro `Pull request`, nuestro trabajo de código en la rama `control-structures-exercises` queda en espera. El siguiente paso es que nuestros revisores revisen el código y nos den el feedback a través de la plataforma de Lyfter.

Aplicar Feedback

Si recibimos alguna solicitud de cambios, no es necesario crear un nuevo `Pull request`. Para subir las modificaciones, vamos a repetir el ciclo de guardado y subida.

Agregamos los archivos actualizados, creamos un nuevo `commit` y hacemos un `push`. Al ejecutar estos comandos, nuestro `Pull request` activo recibirá automáticamente estos nuevos cambios.

```
git add .
git commit -m "Adding edge cases validation based on feedback"
git push
```

Copiar

Hacer `Merge` del Pull Request

Una vez que el entregable sea aprobado, procedemos a ejecutar el `merge` del `PR`. Para ello, ingresamos a la página de nuestro `Pull request`, hacemos clic en el botón verde "*Merge pull request*". Esto unirá los cambios de la rama del ejercicio con la rama destino (`main`).

Imagen referencia para ejecutar el PR 📸

Prepararnos para el siguiente entregable

Si queremos dar inicio a un nuevo entregable, **no** debemos crear la `rama` estando posicionados en la actual.

Para empezar de nuevo, regresamos a la rama principal, descargamos la versión más reciente (*que ya incluye el `PR` que acabamos de mergear*), y creamos una nueva `rama`.

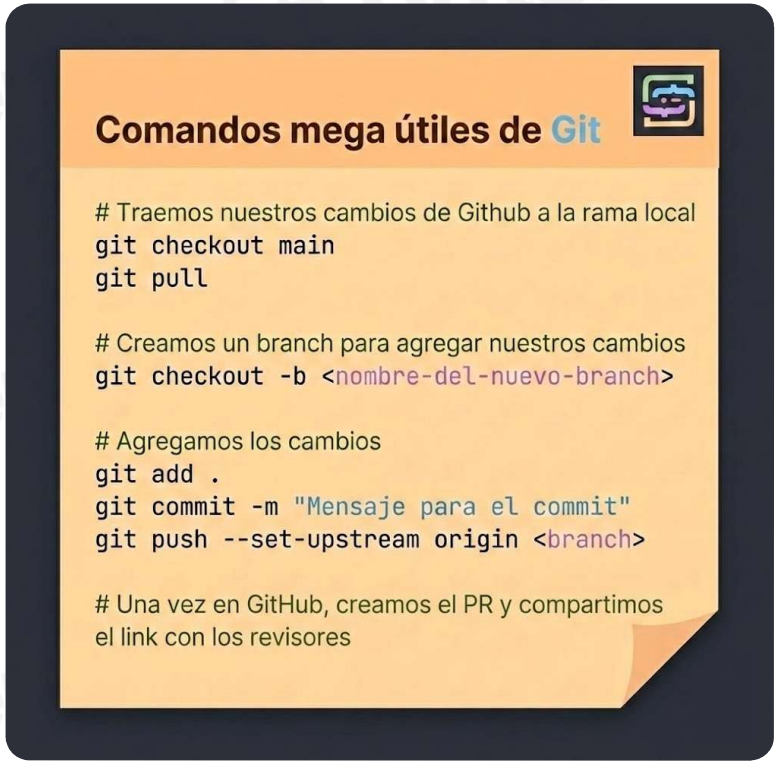
Reiniciamos el flujo desde el paso 4 ejecutando:

```
git checkout main
git pull
git checkout -b new-code-exercise
```

Copiar

Conclusión 🎉

Para que los tengamos siempre a la mano, ya que los vamos a utilizar todos los días como desarrolladores, descarguemos este ***post-it***. Puedes imprimirlo y ¡pegarlo al lado de tu pantalla para nunca olvidarlos!



En esta lección aprendiste:

- Cómo crear un repositorio en `GitHub`.
- Cómo clonarlo en tu máquina local.
- Cómo organizar tus ejercicios en carpetas.
- Cómo usar `git status` para revisar cambios.
- Cómo preparar y hacer tu primer `commit`.
- Cómo subir tus archivos con `git push`.
- Cómo crear un `PR` para recibir feedback.
- Incluir tus nuevos cambios en tu rama principal.

Estos pasos forman la base para trabajar en equipo y mantener un historial limpio y ordenado de tu código. ¡Felicidades por tu primer PR! 🎉